

Database Setup

Use this statement to switch to the **Employee** database before running the remaining SQL commands.

```
USE Employee;
```

Table Creation

Create the **Employee** table to store employee details such as ID, name, email, and department information.

```
CREATE TABLE Employee(  
    EmployeeId int,  
    FirstName Varchar(50),  
    LastName Varchar(50),  
    Email Varchar(50),  
    DepartmentID int,  
    DepartmentName Varchar(50)  
);
```

Create the **Department** table to store department information, including the department ID and department name.

```
CREATE TABLE Department(  
    DepartmentID int,  
    DepartmentName Varchar(50)  
);
```

Insert Sample Data

Use this statement to insert a sample employee record into the **Employee** table.

```
INSERT INTO Employee(  
    EmployeeId,  
    FirstName,  
    LastName,  
    Email,  
    DepartmentID,  
    DepartmentName  
) VALUES(  
    1,  
    'John',  
    'Mathew',  
    'John.mathew@gmailcom',  
    '1',  
    'HR'  
);
```

Use this statement to insert a sample department record into the **Department** table.

```
INSERT INTO Department(  
    DepartmentID,  
    DepartmentName  
) VALUES (  
    '1',  
    'HR'  
)
```

Retrieve Data

Run this query to display all records from the **Employee** table.

```
SELECT * FROM Employee;
```

Run this query to display all records from the **Department** table.

```
SELECT * FROM Department;
```

Stored Procedure

Use this stored procedure to retrieve all records from the **Employee** table in a reusable way.

```
CREATE PROCEDURE GetAllEmployees  
  
AS  
  
BEGIN  
  
    SELECT * FROM Employee;  
  
END
```

IDENTITY(seed, increment)

- **First value (seed)** = Starting number
- **Second value (increment)** = How much to increase each time

Stored Procedures for Table Creation

This stored procedure creates the **Employee** table, using **IDENTITY(1,1)** to generate employee IDs automatically.

```
CREATE PROCEDURE CreateEmployeeTable  
  
AS  
  
BEGIN  
  
    CREATE TABLE Employee(  

```

```
        EmployeeID INT IDENTITY(1,1),
        FirstName VARCHAR(50),
        LastName VARCHAR(50),
        Email VARCHAR(50),
        DepartmentID INT NOT NULL
    );
END;
```

Run this statement to execute the procedure and create the **Employee** table.

```
EXEC CreateEmployeeTable;
```

This stored procedure creates the **Department** table, using **IDENTITY(1,1)** to generate department IDs automatically.

```
CREATE PROCEDURE CreateDepartmentTable
AS
BEGIN
    CREATE TABLE Department(
        DepartmentID INT IDENTITY(1,1),
        DepartmentName VARCHAR(50)
    );
END;
```

Run this statement to execute the procedure and create the **Department** table.

```
EXEC CreateDepartmentTable;
```

Foreign Key Syntax

```
CONSTRAINT FK_Employee_Department  
FOREIGN KEY (DepartmentId)  
REFERENCES Department(DepartmentId)
```

Let's break it down:

1. **CONSTRAINT FK_Employee_Department**

Use **CONSTRAINT** to give a name to the foreign key. **FK_Employee_Department** is the constraint name, and **FK** usually stands for Foreign Key.

You can choose any meaningful name, such as **FK_Emp_Dept** or **Employee_Department_FK**.

2. **FOREIGN KEY (DepartmentId)**

This tells SQL Server that **DepartmentId** in the **Employee** table is a foreign key. In this case, **Employee.DepartmentId** is the child column.

3. **REFERENCES Department(DepartmentId)**

This tells SQL Server that the value stored in **Employee.DepartmentId** must already exist in **Department.DepartmentId**.

Parent table: **Department**

Parent column: **DepartmentId**

Employee and Class Database Example

Use this statement to switch to the **Employee** database before running the following commands.

```
USE Employee;
```

Use these statements to remove existing tables and procedures before recreating them.

```
DROP TABLE Employee;  
DROP TABLE Department;  
DROP PROCEDURE AddDepartment;  
DROP PROCEDURE CreateDepartmentTable;  
DROP PROCEDURE GetAllDepartment;
```

This stored procedure creates the **Department** table with an identity column and a primary key.

```
CREATE PROCEDURE CreateDepartmentTable  
AS
```

```
BEGIN
CREATE TABLE Department(
    DepartmentID INT IDENTITY(1,1) PRIMARY KEY,
    DepartmentName VARCHAR(50)
);
END;
```

Run this statement to execute the procedure and create the **Department** table.

```
EXEC CreateDepartmentTable;
```

This stored procedure creates the **Employee** table and adds a foreign key that links it to the **Department** table.

```
CREATE PROCEDURE CreateEmployeeTable
AS
BEGIN
    CREATE TABLE Employee(
        EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50) NOT NULL,

        DepartmentID INT NOT NULL,

        CONSTRAINT FK_Employee_Department
        FOREIGN KEY (DepartmentID)
        REFERENCES Department(DepartmentID)
    );
END;
```

Run this statement to execute the procedure and create the **Employee** table.

```
EXEC CreateEmployeeTable;
```

Class Database Example

Use these statements to create and switch to the **Class** database.

```
CREATE DATABASE Class;
USE Class;
```

This stored procedure creates the **Class** table with an identity column and a primary key.

```
CREATE PROCEDURE CreateClassTable
AS
BEGIN
    CREATE TABLE Class(
        ClassID INT IDENTITY(1,1) PRIMARY KEY,
        ClassName VARCHAR(50) NOT NULL
    );
END;
```

Run this statement to execute the procedure and create the **Class** table.

```
EXEC CreateClassTable;
```

This stored procedure creates the **Student** table and adds a foreign key that links it to the **Class** table.

```
CREATE PROCEDURE CreateStudentTable
AS
BEGIN
    CREATE TABLE Student(
        StudentID INT IDENTITY(1,1) PRIMARY KEY,
        FirstName VARCHAR(50) NOT NULL,
        LastName VARCHAR(50) NOT NULL,
        Email VARCHAR(50) NOT NULL,

        ClassID INT NOT NULL,

        CONSTRAINT Fk_Class_Student
        FOREIGN KEY (ClassID)
        REFERENCES Class(ClassID)

    );
END;
```

Run this statement to execute the procedure and create the **Student** table.

```
EXEC CreateStudentTable;
```